МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

ФАКУЛЬТЕТ УПРАВЛІННЯ ФІНАНСАМИ ТА БІЗНЕСУ

Кафедра цифрової економіки та бізнес-аналітики

КУРСОВА РОБОТА
з навчальної дисципліни „Проектування та адміністрування БД і СД”

на тему:

Інформаційна система для курсів іноземних мов

Галузь знань: 05 «Соціальні та поведінкові науки»

Спеціальність: 051 «Економіка»

Спеціалізація: «Інформаційні технології в бізнесі»

Освітній ступінь: бакалавр

Науковий керівник:
к.е.н., доцент Старух А. І.
(науковий ступінь, посада, прізвище, ініціали)
_____“___” травня 2022 р.
(підпис)

Виконавець:
студент(ка) групи УФЕ-31 с
Семків О.В.
_____ “___” травня 2022 р.
(підпис)

Загальна кількість балів _____ _____

_____

_____

(підпис, ПІП членів комісії)

ЛЬВІВ 2022

ЗМІСТ

ВСТУП **3**
РОЗДІЛ 1. АНАЛІЗ ВИМОГ **5**
1.1 Постановка завдання **5**
1.2 Розробка моделі варіантів використання веб-сайту **5**
1.3 Аналіз засобів реалізації **7**
РОЗДІЛ 2. РОЗРОБКА БАЗА ДАНИХ **12**
2.1 Опис моделі даних **12**
2.2 Нормалізація відношень **15**
2.3 Визначення типів даних **16**
2.4 Обмеження цілісності даних **19**
2.5 Реалізація SQL скрипта **20**
РОЗДІЛ 3. РОЗРОБКА МАКЕТУ ВЕБ-САЙТУ **24**
3.1 Структура та макет веб-сайту **24**
3.2 Програмування клієнтської частини **28**
ВИСНОВКИ **33**
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ **34**
ДОДАТКИ **35**

ВСТУП

Актуальність теми дослідження. Знання іноземних мов – це ключ до успіху в сучасному світі, де спілкування іноземними мовами та обробка величезних обсягів інформації набуває все більшого значення. Інтерес до вивчення мов традиційно великий, бо перефразовуючи відомий вислів, можна сміливо сказати, що той, хто володіє мовами, володіє світом. Загалом людина, яка володіє мовами, – різнобічно розвинута особистість, володіє кращими здібностями до вивчення нового, вільніша та більш впевнена у спілкуванні з людьми. Зовсім недавно вчені з Лондонського університету шляхом проведення великого статистичного та наукового дослідження прийшли до висновку, що люди, які вчать іноземні мови, мають більш гнучке і оригінальне мислення. Цей висновок базується на тому, що вчені науковим шляхом підтвердили прямий зв'язок між активністю сірої речовини і знанням кількох мов. Стара приказка говорить: «Скільки мов ти знаєш, стільки раз ти людина». Тобто, чим більше мов на сьогодні ти знаєш, тим краще. Адже, кожна мова – це ключ, який відкриває двері до чогось свого, до чогось нового. Так, наприклад, англійська мова – це офіційна мова міжнародного бізнесу та торгівлі, Інтернету і техніки, науки і мистецтв. 80% ділового мовного простору займає саме вона. Кожен з нас все частіше стикається з нею у спілкуванні з партнерами по роботі і на відпочинку. Англійська мова використовується при заповненні анкет, складання резюме, у діловому та приватному листуванні. Володіння англійською мовою – вже не дивовижна навичка, а необхідність. Нині її навчають у дитячих садках, школах, вузах, не кажучи вже про численні курси та тренінги. А в деяких країнах навіть математика і деякі інші предмети шкільної програми викладають англійською мовою.

Отже, ми чітко бачимо, що у наш час з'являється все більше і більше охочих знати іноземні мови. Вони розуміють важливість їх вивчення. Вони не вважають це прагнення лише даниною моді, а замислюються про ті переваги, які дає знання іноземних мов будь-якій культурній людині. Вивчення будь-якої іноземної мови розкриває перед нами нові можливості, робить багатшим наш духовний світ. Володіючи іноземною мовою, людина автоматично переходить на вищий соціальний

ступінь, оскільки стає конкурентоспроможним претендентом на високооплачувану посаду. Адже, як відомо, престижні організації, зазвичай, або самі є представниками іноземних фірм, або ведуть з ними активну партнерську діяльність. А там без знання іноземної мови не обійтися. Іноземні мови покращують якість нашого життя у всіх її сферах.

Мета і завдання дослідження. *Метою курсової роботи* є розробка бази даних і веб-сайту, на якому можна обрати курс і відправити заявку, також можливість опрацьовувати запити, відповідно потреб користувача та адміністартора .

Виходячи з цього, для досягнення мети в роботі поставлено й вирішено такі теоретичні та практичні *завдання*:

• охарактеризувати інформаційну систему для курсів іноземних мов;

• проаналізувати предметну область;

• розробити архітектуру та спроектувати структуру бази даних;

• розробити інформаційну систему(веб-сайт), що дає змогу переглянути перелік курсів та подати онлайн заявку на курс.

Об'єкт дослідження є особливість розробки інформаційної системи для курсів іноземних мов за допомогою системи управління базами даних MYSQL.

Предмет досліджень є теоретичні, методичні та практичні аспекти розробки програмного забезпечення за допомогою різних мовою проектування.

Використане програмне забезпечення. Для досягнення результату використовувалось середовище розробки Visual Studio Code та мови програмування: HTML, CSS, JS.

Структура роботи. Курсова робота складається з трьох розділів («Аналіз вимог», «Розробка бази даних» та «Розробка веб-додатку»), висновків, списку використаних джерел та додатків. Загальний обсяг роботи − 59 сторінок.

РОЗДІЛ 1. АНАЛІЗ ВИМОГ

1.1 Постановка завдання

Головні ідеї сучасних інформаційних технологій базуються на концепції баз даних. Відповідно до цієї концепції, основою інформаційних технологій є дані, які повинні бути організовані в бази даних для ефективної роботи з інформацією.

Інформація є одним з найцінніших ресурсів суспільства. Інформаційна технологія передбачає вміння грамотно працювати з інформацією і обчислювальною технікою. У свою чергу, дані - це інформація, представлена в певному вигляді, що дозволяє автоматизувати її збір, зберігання і подальшу обробку людиною або інформаційним засобом. Для комп'ютерних технологій дані - це інформація в дискретному, фіксованому вигляді, зручна для зберігання, обробки, а також для передачі по каналах зв'язку.

Інформаційна система курсів іноземних мов призначена в першу чергу для того щоб користувач міг залишити заявку на обраний курс. Будь-який користувач сайту має змогу переглядати інформацію про курси. В його розпорядженні є форма одностороннього зв’язку — запит. Заповнивши всі обов’язкові поля які вказані в заявці, дані потрапляють до керівництва, яке в найкоротші терміни відреагує на звернення.

1.2 Розробка моделі варіантів використання веб-сайту

Інформаційна система має 2 елементи керування:

- Користувач
- Адміністратор

Користувач має змогу переглядати головну сторінку, переглядати інформацію про кожен курс детально. Також користувач може переглянути інформацію про викладачів. Користувач має можливість залишити заявку за допомогою модального

вікна при виборі курсу. Користувач також може переглядати розклад занять за весь тиждень або детально за кожен день. Головний користувач інформаційної системи – це адміністратор. Він може змінювати існуючу інформацію або додавати нову. Адміністратор має змогу додавати нові курси або видаляти існуючі, аналогічно це працює з викладачами. Також адміністратор має можливість детально аналізувати статистику курсів та статистику викладачів. Адміністратор може відслідковувати клієнтів які здійснили покупки на суму більше 10000, для того щоб нарахувати знижку. Також аналогічно працює з викладачами які викладають курси на суму більше 7000, для них також нараховуються бонус в вигляді премії. Адміністартор має можливість віслідковувати детальну статистику по продажу курсів і скільки прибутку получено за їх продаж.

Рис. 1.1 Діаграма варіантів використання для користувача

В подальшому система може збільшувати функціональний потенціал.

1.3 Аналіз засобів реалізації

HTML (Hypertext Markup Language) – це мова розмітки, яка використовується для структурування і відображення веб-сторінки і її контенту. Наприклад, контент може бути структурованим всередині безлічі параграфів, маркованих списків секцій або таблиць. Важливо розуміти що HTML не є мовою програмування; це мова розмітки, і використовується, щоб повідомляти браузеру, як відображати веб-сторінки, які відвідує користувач. Він може бути складним або простим, в залежності від того, як хоче веб-дизайнер. HTML складається з ряду елементів - тегів, які використовуються, щоб вкладати або обертати різні частини контенту, щоб змусити контент відображатися або діяти за певним чином. Теги можуть зробити слово або зображення посиланням на щось, зробити слова курсивом або змінити розмір шрифту. Наприклад, візьмемо наступну стрічку контенту :

«JavaScript – це мова програмування, яка використовується при створенні сайтів»

Якби потрібно було, відокремити рядок, можна було б вказати що це абзац, укладаючи його в теги абзацу:

 «JavaScript – це мова програмування, яка використовується при створенні сайтів»

Головними частинами елементу є:

1. **Відкриваючий тег** (Opening tag): Складається з імені елемента (в даному випадку, "p"). Він вказує на те, де елемент починається або починає діяти, в даному випадку - де починається абзац.
2. **Закриваючий тег** (Closing tag): Це те ж саме, що і відкриваючий тег, за винятком того, що він включає в себе косу риску перед іменем елемента. Закриваючий тег вказує, на те де елемент закінчується.
3. **Контент** (Content): Це контент елемента, який в даному випадку є просто текстом.
4. **Елемент** (Element): Відкриваючий та закриваючий теги і контент разом складають елемент.

Кожна HTML сторінка має так званий «скелет» , який потрібний для її функціонування:

```
<! DOCTYPE html>
<html>
  <head>
    <meta charset = "utf-8">
    <title> Document</ title>
  </ head>
  <body>
 </ body>
</ html>
```

1. Елемент **<! DOCTYPE html >** призначений для вказівки типу поточного документа - DTD (document type definition, опис типу документа). Це необхідно, щоб браузер розумів, як слід інтерпретувати поточну веб-сторінку.
2. **<html> </ html>** - елемент <html>. Цей елемент обертає весь контент на всій сторінці, і іноді відомий як кореневий елемент.
3. **<head> </ head>** - елемент <head>. Цей елемент виступає в якості контейнера для всього, що ви побажаєте включити на HTML сторінку, але не є контентом, який показується користувачам сторінки.
4. **<body> </ body>** - елемент <body>. У ньому міститься весь контент, який показується користувачам, коли вони відвідують сторінку, будь то текст, зображення, відео або щось ще.
5. **<meta charset = "utf-8">** - цей елемент встановлює UTF-8 кодування документа, яка включає в себе більшість символів з усіх відомих людських мов.
6. **<title> </ title>** - елемент <title>. Цей елемент встановлює заголовок для веб-сторінки сторінки

CSS

CSS (Cascading Style Sheets, каскадні таблиці стилів) - мова опису зовнішнього вигляду HTML-документа. Це одна з базових технологій в сучасному інтернеті. Практично жоден сайт не обходиться без CSS, тому
HTML і CSS працюють в одній зв'язці.

Каскадні таблиці стилів працюють з HTML, але це зовсім інша мова. HTML структурує документ і впорядковує інформацію, а CSS взаємодіє з браузером, щоб надати документу оформлення.

Для чого використовується CSS?

По суті, CSS - це файл, де описується, як буде виглядати кожен з елементів на сторінці. За HTML, таким чином, залишається тільки структура сторінки: самі блоки, їх вміст і розташування. Створити сторінку і оформити її можна і без використання CSS прописуючи візуальні властивості кожного елемента в його базових стилях. Але, якщо сторінок сотні і тисячі, застосовувати такий метод незручно: при зміні оформлення доводиться міняти безліч документів, до того ж це засмічує верстку. Тому використання CSS вважається золотим стандартом оформлення сайтів: так вийшло завдяки гнучкості і різноманіттю можливостей каскадних таблиць.

Переваги CSS

- Це відчутно спрощує верстку і знижує затрати на час. Один створений файл стилів можна поширити на безліч сторінок, так що зовнішній вигляд елементів досить описати один раз.
- Якщо щось потрібно змінити, достатньо внести правки в один файл. Це стосується і змін в оформленні, і знайдених помилок.
- Застосування CSS серйозно полегшує структуру документа, що добре і для користувачів, і для пошукових програм.
- Варіативність оформлення збільшується. CSS підтримує набагато більше можливостей, ніж є при використанні чистого HTML, на додачу тут можна

робити так звану «адаптивну верстку», коли стилі сторінки змінюються відносно розмірів екрану користувача.

- Сторінки починають завантажуватися швидше: браузер кешує CSS при першому відвідуванні сайту, при наступних завантажуються тільки дані, що набагато швидше.

Синтаксис та структура

Файл CSS зводиться до набору правил, описаних за певним синтаксисом. Правило складається з селекторної частини і блоку оголошень: ними описуються всілякі елементи сторінки. Приклад правила:

Вся структура називається набором правил (але часто для стислості "правило"). Розбір окремих частин:

Селектор (Selector)

Спосіб вибору HTML елемента на початку правила. Він вибирає елемент (и) для застосування стилю (в даному випадку, елементи p). Для стилізації іншого елемента, просто заміняється селектор.

Властивості (Properties)

Способи, якими стилізується певний HTML елемент (в даному випадку, color є властивістю для елементів <p>).

Оголошення (Declaration)

Єдине правило, наприклад color: red; вказує, які з властивостей елемента стилізуються.

Значення властивості (Property value)

Праворуч від властивості, після двокрапки, є значення властивості, яке приймає одне з безлічі можливих ознак для даної властивості (існує безліч значень color, крім red).

Важливі частини синтаксису:

- Кожен набір правил (крім селектора) повинен бути обгорнутий в фігурні дужки ({}).
- У кожному оголошенні необхідно використовувати двокрапку (:), щоб відокремити властивість від його значень.
- У кожному наборі правил треба використовувати крапку з комою (;), щоб відокремити кожне оголошення від наступного.

Різні типи селекторів у CSS

Типи селекторів у CSS

Таблиця 1.1

Ім'я селектора	**Що вибирає**	**Приклад**
Селектор тегу	Всі HTML елемент (и) з вибраним тегом	**селектор p** вибирає елементи з тегом <p>
ID селектор	Елемент на сторінці з визначеним унікальним ID на даній HTML.	**#my-id** вибирає <p id="my-id">
Селектор класу	Елемент (и) на сторінці з визначеним класом.	**.my-class** Вибирає <p class="my-class"> та <a class="my-class">
Селектор атрибута	Елемент (и) на сторінці з зазначеним атрибутом .	**img[src]** Выбирає <img src="myimage.png"> але не <img>

РОЗДІЛ 2. РОЗРОБКА БАЗА ДАНИХ

2.1 Опис моделі даних

База даних — це певний набір даних, які пов'язані між собою спільною ознакою або властивістю, та впорядковані, наприклад, за алфавітом.
Об'єднання великої кількості даних в єдину базу дає змогу для формування безлічі варіації групування інформації — особисті дані клієнта, історія замовлень, каталог товарів та будь-що інше.

Головною перевагою БД є швидкість внесення та використання потрібної інформації. Завдяки спеціальним алгоритмам, які використовуються для баз даних, можна легко знаходити необхідні дані всього за декілька секунд. Також в базі даних існує певний взаємозв'язок інформації: зміна в одному рядку може спричинити зміни в інших рядках — це допомагає працювати з інформацією простіше і швидше.

Бази даних для сайтів дають змогу зберігати інформацію, що виглядає як зв'язані між собою таблиці. Саме в БД зберігаються вся необхідна та корисна інформація для функціонування сайту (клієнтські дані, прайс-лист, список товарів).

Щоб створити запит до бази даних часто використовують Structured Query Language. SQL дає змогу додавати, редагувати та видаляти інформацію, що міститься у таблицях.
Під час програмування сайтів використовують різні системи управління БД. До основних СУБД, відносять:

- об'єктно-реляційна система управління базами даних Oracle Database;
- вільна система управління базами даних PostgreSQL;
- система керування базами даних Microsoft SQL Сервер;
- вільна система управління базами даних MySQL;

Такі системи управління відрізняються централізованою обробкою запитів, забезпечують надійність, доступність та безпеку БД.

Найбільш популярною системою управління є MySQL, вона дає зручний доступ для управління БД та підтримує велику кількість таблиць різних типів.

При інформаційної системи було розроблено реляційну базу даних. Для візуального відображення структури створено ER – діаграму (рис.2.1)

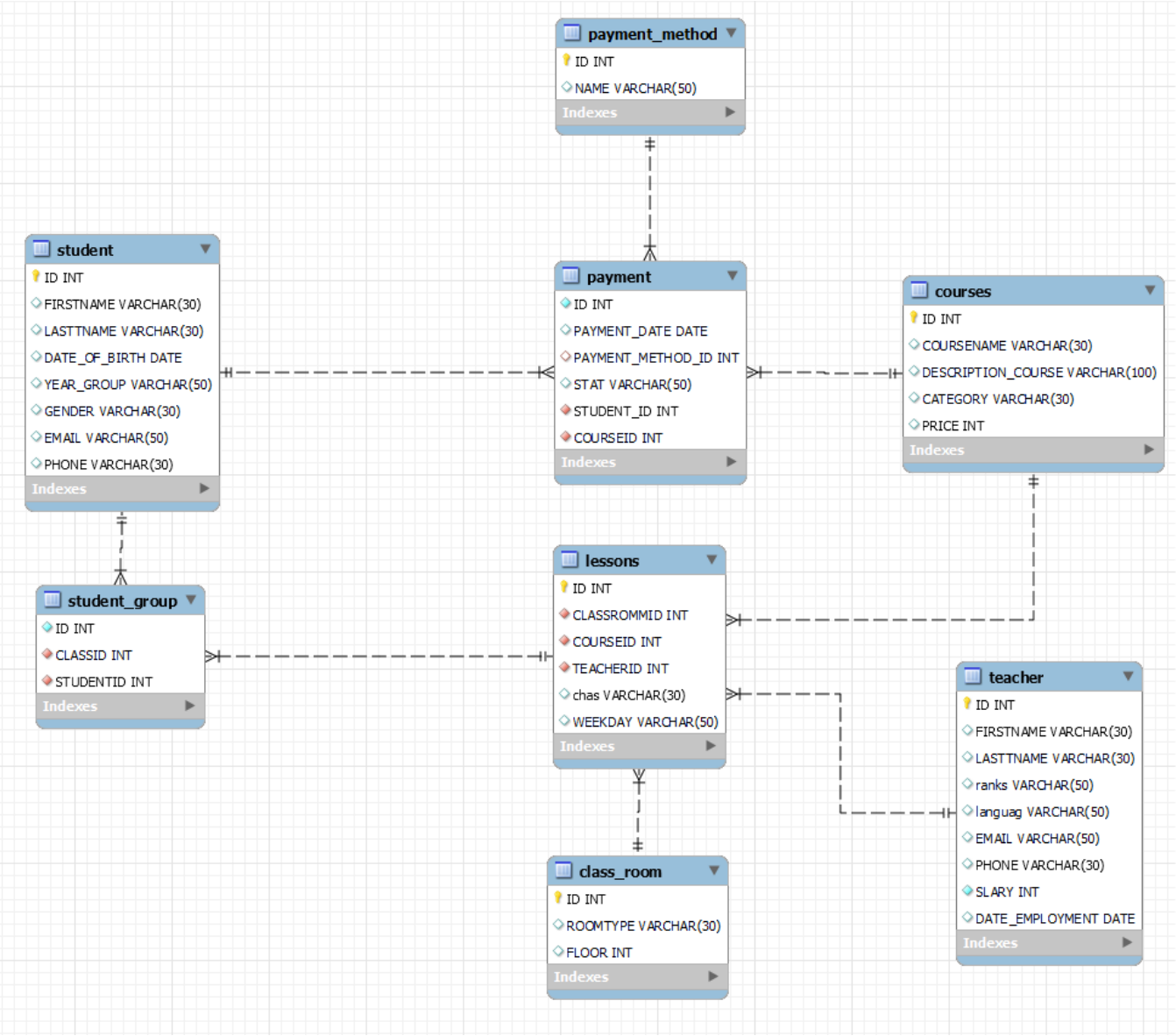

Рис. 2.1 Діаграма сутностей та зв'язків

На рисунку (Рис.2.1) можна побачити, що дана модель складається з восьми таблиць: lessons, class_room, teacher, student_group, courses, payment, payment_method, student.

Lessons таблиця загальна яка містить інформацію про курс:

a) ID – Унікальне значення кожного запису в таблиці;

b) CLASSROMMID – Унікальне значення class_room;

c) COURSEID – Унікальне значення курсу;

d) TEACHERID – Унікальне значення викладача;

e) CHAS – Година заняття;

f) WEEKDAY – День тижня;

Class_Room таблиця яка містить інформацію про навчальний кабінет:

a) ID – Унікальне значення кожного запису в таблиці;
b) ROOMTYPE – тип кімнати;
c) FLOOR – Поверх.

Teacher таблиця яка містить інформацію про викладачів:

a) ID – Унікальне значення кожного запису в таблиці;
b) FIRSTNAME – Ім'я викладача;
c) LASTNAME – прізвище викладача;
d) RANKS – Рівень знань викладача;
e) LANGUAG – Мова викладання;
f) EMAIL – Електронна пошта викладача;
g) PHONE – Номер викладача;
h) SALARY – Заробітня плата;
i) DATA_EMPLOYEMNT – Дата початку роботи;

Student_group таблиця яка містить інформацію про студента та групу:

a) ID – Унікальне значення кожного запису в таблиці;
b) CLASSID – Значення поля з таблиці LESSONS
c) STUDENTID – Значення поля з таблиці STUDENT

Courses таблиця яка містить інформацію про курси:

a) ID – Унікальне значення кожного запису в таблиці;
b) COURSENAME – Назва курсу;
c) DESCRIPTION_COURSE – Опис курсу;
d) CATEGORY – Категорія курсу;
e) PRICE – Ціна курсу;

Payment таблиця яка містить інформацію про оплату:

a) ID – Унікальне значення кожного запису в таблиці;
b) PAYMENT_DATE – Дата проведення оплати;
c) PAYMENT_METHOD_ID – Унікальне поле з таблиці PAYMENT_METHOD
d) STAT – Статус оплати;
e) STUDENT_ID – Унікальний ідентифікатор студента;
f) COURSEID - Унікальний ідентифікатор обраного курсу;

Payment_method таблиця яка містить методи оплати;

a) ID – Унікальне значення кожного запису в таблиці;
b) NAME – Метод оплати;

2.2 Нормалізація відношень

Нормалізація схеми бази даних — покроковий процес розбиття одного відношення відповідно до алгоритму нормалізації на декілька відношень на базі функціональних залежностей.

Нормальна форма — властивість відношення в реляційній моделі даних, що характеризує його з точки зору надмірності, яка потенційно може призвести до

логічно помилкових результатів вибірки або зміни даних. Нормальна форма визначається як сукупність вимог, яким має задовольняти відношення.

2.3 Визначення типів даних

Структура MySQL трирівнева: бази даних – таблиці – записи. Бази даних і таблиці. Ім'я бази даних MySQL унікальна в межах системи, а таблиці - в межах бази даних, поля - в межах таблиці. База даних з точки зору MySQL - це звичайний каталог, що містить файли певного формату - таблиці. Таблиці складаються із записів, а записи, у свою чергу, складаються з полів. Поле має два атрибути — ім'я і тип даних.

Основні типи даних, які використовувались:

- VARCHAR – може зберігати не більше 255 символів. На відміну від CHAR, для зберігання значення даного типу виділяється необхідна кількість пам'яті;
- INT – діапазон від -2 147 483 648 до 2 147 483 647;
- DATE — Формат: YYYY-MM-DD.

Таблиця 2.1

Склад таблиці Lessons

№	Ім’я атрибуту	Тип даних	Розмір поля
1	ID	INT	11
2	CLASSROMMID	INT	11
3	COURSEID	INT	11
4	TEACHERID	INT	11
5	CHAS	VARCHAR	30
6	WEEKDAY	VARCHAR	50

Таблиця 2.2

Склад таблиці Class_room

№	Ім’я атрибуту	Тип даних	Розмір поля
1	ID	INT	11
2	ROOMTYPE	VARCHAR	30
3	FLOOR	INT	11

Таблиця 2.3

Склад таблиці Teacher

№	Ім’я атрибуту	Тип даних	Розмір поля
1	ID	INT	11
2	FIRSTNAME	VARCHAR	30
3	LASTNAME	VARCHAR	30
4	RANKS	VARCHAR	50
5	LANGUAG	VARCHAR	50
6	EMAIL	VARCHAR	50
7	PHONE	VARCHAR	30
8	SALARY	INT	11
9	DATA_EMPLOYEMNT	DATE	9999

Таблиця 2.4

Склад таблиці Student_group

№	Ім’я атрибуту	Тип даних	Розмір поля
1	ID	INT	11
2	CLASSID	INT	11
3	STUDENTID	INT	11

Таблиця 2.5

Склад таблиці Courses

№	Ім’я атрибуту	Тип даних	Розмір поля
1	ID	INT	11
2	COURSENAME	VARCHAR	50
3	DESCRIPTION	VARCHAR	50
4	CATEGORY	VARCHAR	50
5	PRICE	INT	11

Таблиця 2.6

Склад таблиці Payment

№	Ім’я атрибуту	Тип даних	Розмір поля
1	ID	INT	11
2	PAYMENT_DATE	INT	11
3	PAYMENT_METHOD_ID	INT	11
4	STAT	VARCHAR	50
5	STUDENT_ID	INT	11
6	COURSEID	INT	11

Таблиця 2.7

Склад таблиці Payment_method

№	Ім’я атрибуту	Тип даних	Розмір поля
1	ID	INT	11
2	NAME	VARCHAR	50

2.4 Обмеження цілісності даних

Цілісності бази даних — це правила, які дають змогу уникнути введення некоректних даних у БД, а також забезпечити можливість зв'язування декількох таблиць. Ці правила можуть бути описані при створенні чи модифікації таблиці.

В sql виділяють такі пити цілісності даних:

- CHECK — Контроль допустимих значень атрибутів;
- NOT NULL/NULL — Заборона/ дозвіл на використання не заданих або не визначених значень.
- UNIQUE — Контроль унікальності значень атрибутів.
- PRIMARY KEY — Первинний ключ.
- FOREIGN KEY — Зовнішній ключ.

PRIMARY KEY (PK) використовується для ідентифікації рядків таблиці, в нього є певні особливості:

- В одній таблиці БД може бути тільки один PK.
- Рядки, які мають це правило, не можуть мати не визначені або не задані дані. FOREIGN KEY (FG) використовується для того, щоб зв'язать дві таблиці, в нього є такі особливості :
- Поле у головній таблиці, на яке буде адресуваться FG повинне бути оголошене PK або UNIQUE.
- Поле у головній таблиці і поле з якої відбувається адресація повинні мати один й теж самий тип даних

При роздобці даної бази даних „Courses” було використано: primary key, foregion key.

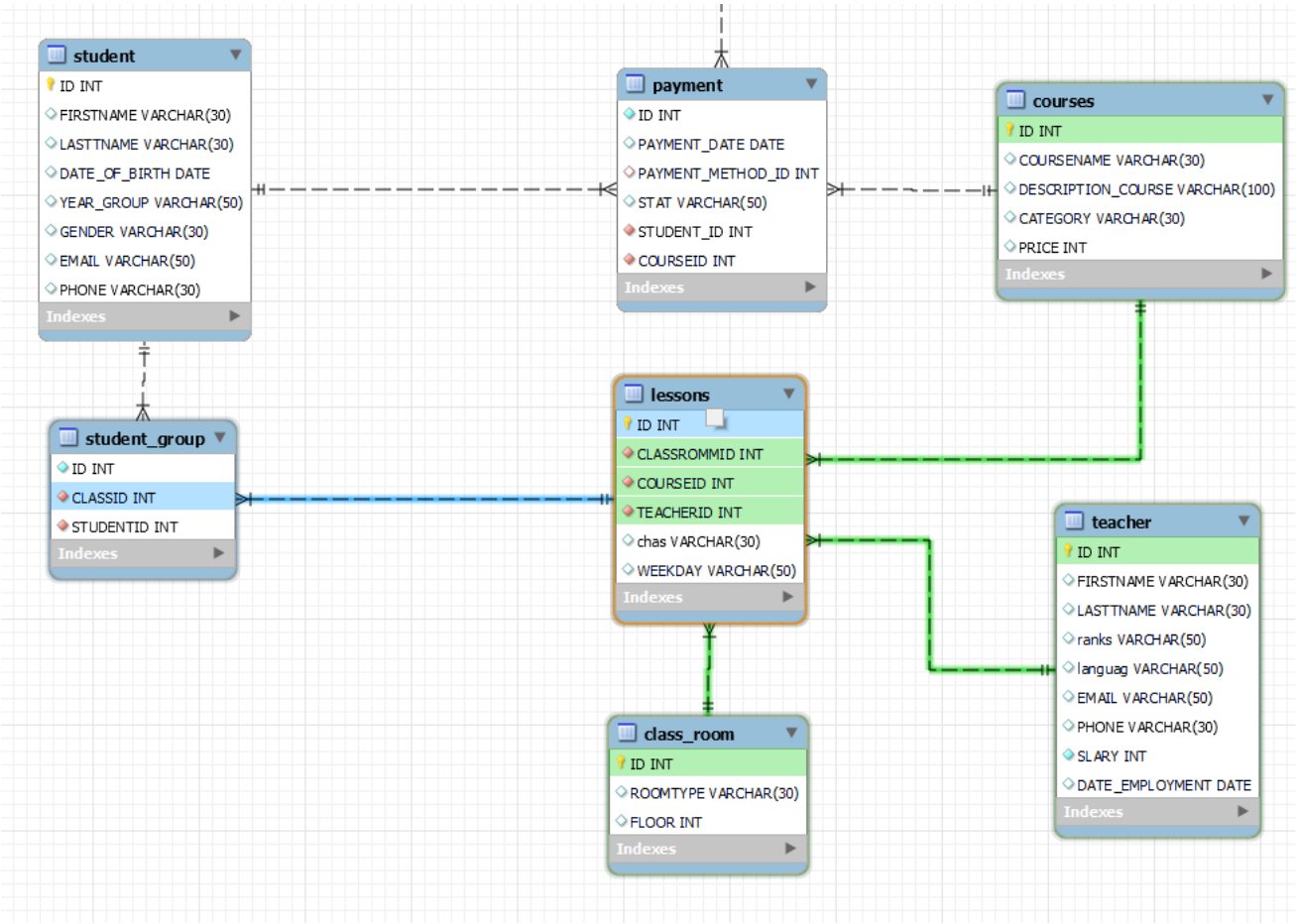

Рис. 2.2 Діаграма сутностей та зв’язків, створений за допомогою PRIMARY KEY і FOREIGN KEY

2.5 Реалізація SQL скрипта

База даних реалізує такі запити:

- Запит який відображає детальний розклад занять:

SELECT LESSONS.ID, teacher.firstname, teacher.lasttname, COURSENAME, LESSONS.weekday, lessons.chas, CLASS_ROOM.ROOMTYPE, CLASS_ROOM.FLOOR FROM LESSONS LEFT JOIN teacher on lessons.teacherid = teacher.id

LEFT JOIN COURSES ON LESSONS.COURSEID = COURSES.ID

LEFT JOIN CLASS_ROOM ON LESSONS.CLASSROMMID = CLASS_ROOM.ID ;

ID	firstname	lasttname	COURSENAME	weekday	chas	ROOMTYPE	FLOOR
1	Denys	Malushko	Курс Англійської ОНЛАЙН	Понеділок	12-00	online	0
2	Olena	Meshenina	Курс Німецької ОНЛАЙН	Вівторок	11-00	online	0
3	Oleg	Maletskuy	Курс Японської ОНЛАЙН	Середа	15-00	online	0
4	Julia	Teslyuk	Курс Іспанської ОНЛАЙН	Четвер	13-00	online	0
5	Nazar	Teslyuk	Курс Української ОНЛАЙН	Пятниця	18-00	online	0
6	Denys	Malushko	Курс Англійської	Понеділок	14-00	conference	1
7	Olena	Meshenina	Курс Німецької	Вівторок	13-00	lesson_room	2
8	Oleg	Maletskuy	Курс Японської	Середа	17-00	conference	3
9	Julia	Teslyuk	Курс Іспанської	Четвер	16-00	conference	5

Рис. 2.3 Результат виконання запиту на розклад занять

- Запит який показує яким способом оплати скористались клієнти:

SELECT PAYMENT.ID, STUDENT.firstname, STUDENT.lasttname, PAYMENT_METHOD.NAME FROM PAYMENT LEFT JOIN STUDENT on PAYMENT.STUDENT_ID = STUDENT.id

LEFT JOIN PAYMENT_METHOD ON PAYMENT.PAYMENT_METHOD_ID = PAYMENT_METHOD.ID WHERE PAYMENT_METHOD_ID = 1;

SELECT PAYMENT.ID, STUDENT.firstname, STUDENT.lasttname, PAYMENT_METHOD.NAME FROM PAYMENT LEFT JOIN STUDENT on PAYMENT.STUDENT_ID = STUDENT.id

LEFT JOIN PAYMENT_METHOD ON PAYMENT.PAYMENT_METHOD_ID = PAYMENT_METHOD.ID WHERE PAYMENT_METHOD_ID = 2;

ID	firstname	lasttname	NAME
1	Oleh	Semkiv	CASH
2	Oleh	Semkiv	CASH
3	Oleh	Semkiv	CASH
4	Oleh	Semkiv	CASH
5	Bohdan	Felypets	CASH
6	Bohdan	Felypets	CASH
7	Yaroslav	Yarynych	CASH
8	Yaroslav	Yarynych	CASH
9	Yaroslav	Yarynych	CASH
10	Yaroslav	Yarynych	CASH
11	Sergiy	Tekh	CASH
12	Maria	Lenko	CASH
13	Roman	Kuzma	CASH
14	Julia	Bil	CASH
15	Yaroslav	Mudryi	CASH
16	Oleg	Barylko	CASH

Рис. 2.4 Результат виконання запиту на спосіб оплати

- Запит який показує фільтрацію курсів по ціні (по спаданню і зростанню)

SELECT *FROM courses ORDER BY price DESC;

SELECT *FROM courses ORDER BY price asc;

ID	COURSENAME	DESCRIPTION_COURSE	CATEGORY	PRICE
9	Курс Іспанської	SPANISH_course_description	OFFLINE	15000
8	Курс Японської	JAPAN_course_description	OFFLINE	12000
6	Курс Англійської	ENGLISH_course_description	OFFLINE	10000
19	Курс Іспанської Граматика	SPANISH_course_description	OFFLINE	10000
10	Курс Української	UKRAINE_course_description	OFFLINE	9500
7	Курс Німецької	germany_course_description	OFFLINE	9000
4	Курс Іспанської ОНЛАЙН	SPANISH_course_description	ONLINE	7800
3	Курс Японської ОНЛАЙН	JAPAN_course_description	ONLINE	6000
16	Курс Англійської Граматика	ENGLISH_course_description	OFFLINE	6000
18	Курс Японської Граматика	JAPAN_course_description	OFFLINE	6000
1	Курс Англійської ОНЛАЙН	ENGLISH_course_description	ONLINE	5000
12	Курс Німецької Розмовний	GERMANY_course_description	ONLINE	5000
13	Курс Японської Розмовний	JAPAN_course_description	ONLINE	5000
17	Курс німецької Граматика	germany_course_description	OFFLINE	5000
20	Курс Української Граматика	UKRAINE_course_description	OFFLINE	5000
5	Курс Української ОНЛАЙН	UKRAINE_course_description	ONLINE	4750
2	Курс Німецької ОНЛАЙН	GERMANY_course_description	ONLINE	4500

Рис. 2.5 Результат виконання запиту на фільтрацію цін

- Запит який показує студентів які підходять для нарахування знижки

SELECT STUDENT.ID, STUDENT.FIRSTNAME, STUDENT.LASTTNAME, COURSES.price as sum

FROM STUDENT left join PAYMENT on PAYMENT.STUDENT_ID = STUDENT.ID

left JOIN COURSES ON PAYMENT.COURSEID = courses.id

group by STUDENT.id having sum(price) > 10000;

ID	FIRSTNAME	LASTTNAME	sum
1	Oleh	Semkiv	5000
3	Bohdan	Felypets	6000
4	Yaroslav	Yarynych	4750
6	Maria	Lenko	12000
7	Roman	Kuzma	15000
16	Stepan	Struk	12000
19	Yevgen	Osoba	12000
26	Olha	Koval	4750
27	Roman	Petrakov	15000

Рис. 2.5 Результат виконання запиту на нарахування знижки

- Запит який показує викладачів які підходять для нарахування премії

SELECT TEACHER.ID, TEACHER.FIRSTNAME, TEACHER.LASTTNAME, COURSES.price as sum

FROM TEACHER left join LESSONS on LESSONS.TEACHERID = TEACHER.ID

left JOIN COURSES ON LESSONS.COURSEID = courses.id

group by TEACHER.id having sum(price) > 7000;

ID	FIRSTNAME	LASTTNAME	sum
1	Denys	Malushko	5000
2	Olena	Meshenina	4500
3	Oleg	Maletskuy	6000
4	Julia	Teslyuk	7800

Рис. 2.6 Результат виконання запиту на нарахування премії

- Запит на перегляд загальної статистики по продажу курсів

select courses.id as 'АЙДІ' , courses.coursename as КУРС, count(payment.id) as КІЛЬКІСТЬ, SUM(COURSES.PRICE) as СУМА

from courses join payment on courses.id = payment.courseid

where payment.stat like 'COMPLETE'

GROUP BY COURSES.ID order by СУМА;

АЙДІ	КУРС	КІЛЬКІСТЬ	СУМА
2	Курс Німецької ОНЛАЙН	1	4500
7	Курс Німецької	1	9000
5	Курс Української ОНЛАЙН	2	9500
10	Курс Української	1	9500
3	Курс Японської ОНЛАЙН	2	12000
6	Курс Англійської	2	20000
4	Курс Іспанської ОНЛАЙН	3	23400
8	Курс Японської	2	24000
9	Курс Іспанської	3	45000

Рис. 2.7 Результат виконання запиту на статистику продажу курсів

РОЗДІЛ 3. РОЗРОБКА МАКЕТУ ВЕБ-САЙТУ

3.1 Структура та макет веб-сайту

Структура сайту — це схема розташування його сторінок, категорій, підкатегорій і товарів. Це своєрідний план, в якому прослідковується логічний зв'язок між сторінками. З технічної точки зору навігація ресурсу являє собою набір URL, що розташовані в певній послідовності. Вона нерозривно пов'язана з семантичним ядром. А саме воно визначає, які папки та документи мають бути на сайті.

Важливо також розрізняти, що структура сайту є зовнішня і внутрішня. Під першою розуміють макет сторінки із зазначенням розташування на ній блоків. Друга відображає категорії, належність до них певних сторінок і матеріалів. Саме про неї піде мова далі.

Типи структур веб-сайтів

Якщо у вас є семантичне ядро і ви розумієте, які саме матеріали мають бути на сайті, повинна вимальовуватись структура ресурсу у вигляді схеми. Іншими словами — ієрархія, логічний та послідовний ланцюжок побудови й подачі інформації.

Якого виду може бути схема, залежить від мети сайту. Виходячи з цього обирають найбільш відповідний тип структури:

1. Алфавітна організація. Передбачає розташування інформації в алфавітному порядку. Зручна, коли клієнт точно знає назву товару, який шукає.
2. Хронологічна. Найчастіше використовують при розміщенні новин, прес-релізів, що дозволяє орієнтуватися за датою виходу потрібного матеріалу.

3. Деревоподібна. Коли є головна сторінка, а від неї вже йдуть різні категорії, картки товарів тощо. Це популярна структура інтернет магазину, розробкою якої потрібно займатися дуже ретельно.
4. Тематична. Зручна для користувачів навігація, коли вся інформація розташована за темами.
5. Лінійна. У такому випадку вся інформація розташовується у вигляді ланцюжка зі своїми компонентами. У людини тут немає можливості переміщатися зі сторінки на сторінку як їй заманеться, все відбувається за конкретно заданим маршрутом. Це типовий варіант для візитки.
6. Гратчаста. За неї всі складові розміщуються в окремих гілках. Її частіше можна зустріти в каталогах інформаційних статей.
7. Павутинна. Це комбінована структура, яка передбачає поєднання в собі декількох типів. Однак через свою складність вона застосовується нечасто.
8. Гібридна. Також передбачає комбінацію декількох типів структур, що дозволяє користувачам простіше та швидше знаходити потрібну інформацію.

Ієрархічна структура веб-сайту допомагає користувачам швидко знайти товар, який їх цікавить, не витрачаючи не це зайвого часу.

Особливості проектування структури сайту

Можна знайти свої за і проти для кожного типу структури. Проте для максимальної ефективності розробка ресурсу має відбуватись з урахуванням наступних правил:

1. Навігація має бути якомога зрозумілішою для клієнта.
2. Усі сторінки каталогу слід робити максимально оптимізованими і логічними.
3. Користуватися сайтом повинно бути просто пересічному громадянину.

Серед найбільш типових помилок можна виділити наступні:

- постійне створення нових сторінок;
- дублювання товарів за назвою чи змістом;
- розміщення товарів із підкатегорій в категоріях;

- незаповнені сторінки чи категорії, в яких знаходиться дуже мало товарів;
- нерівномірний розподіл товарів у групах;
- особливості структури веб-сайту передбачають розміщення товарів і комплектуючих на одній сторінці;
- обмежена кількість фільтрів або параметрів;
- назви категорій збігаються з назвами брендів.
-

Створення макету сайту — процес складний і довготривалий. Важливо розуміти, що для інтернет-магазину потрібна зовсім інша структура, ніж для комерційного ресурсу компанії чи сайту візитки. До процесу створення відносяться наступні етапи:

- проектування семантичної основи (визначення семантичного ядра);
- групування категорій (систематизація товарів);
- визначення пріоритетів у навігації (ієрархічне розміщення категорій);
- закриття потреб різних груп цільової аудиторії (основні складові структури сайту мають закрити потреби кожної групи, адже на багатьох ресурсах вона не одна);
- додавання можливості масштабування (для подальшого розвитку варто залишити можливість розширення).

Доступ до сайту відбувається через адресу: http://127.0.0.1:5500/index.html. Структуру сайту складається з таких елементів:

- Хедер. Містить логотип, кнопки для переходу по розділах та номер теелфону (Додаток А)
- Фронтальна секція. Містить заставку та кнопку для переходу до вибору курсів (Додаток Б)

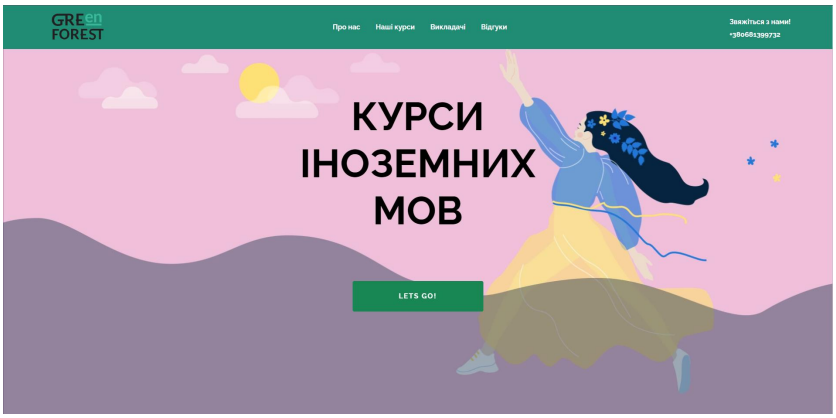

Рис. 3.1 Фронтальна секція

- Секція «Наші переваги» містить інформацію про собливості та унікальність курсів (Додаток В)
- Перелік доступних курсів. Містить список доступних курсів та кнопки для покупки курсу

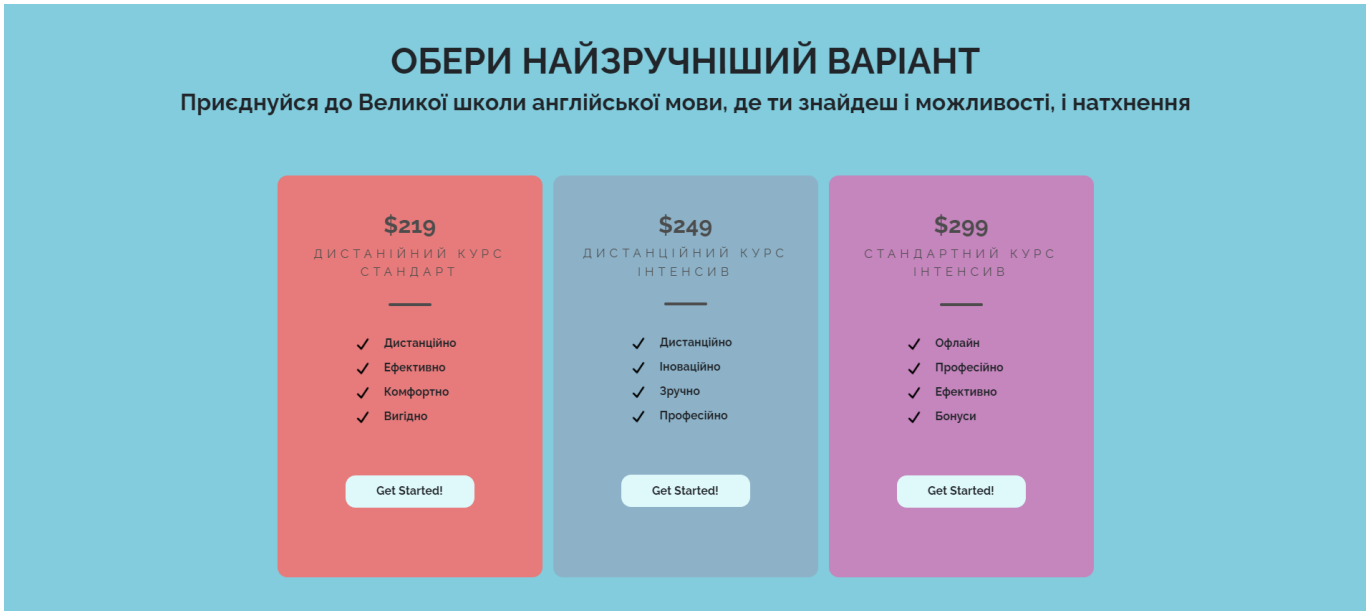

Рис. 3.2 Перелік доступних курсів

- Секція «Про нас». Містить детальну інформацію про компанію. (Додаток Ґ)
- Секція «Наші курси». Розклад та деталі по кожному курсу. (Додаток Д)
- Секція «Наші викладачі». Містить інфомрацію про перелік викладачів (Коротка біографія. Стаж, особливості) (Додаток Е)
- Секція «Відгуки» містить відгуки студентів які закінчили курс навчання. (Додаток Є)

- Секція «Чому варто обрати нас ?» . Містить перелік переваг з меню-баром. (Додаток Ж)
- Футер. Дублює хедер та містить посилання на соціальні мережі, також містить карту. (Додаток З)

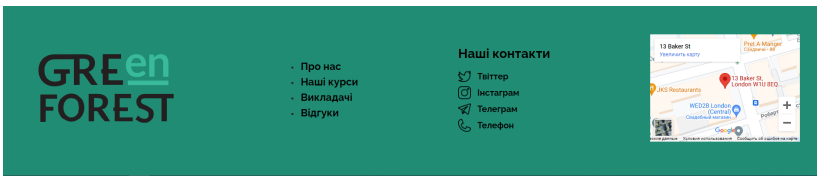

Рис. 3.3 Футер сайту

3.2 Програмування клієнтської частини

Для розробки проекту було використано середовище Visual Studio Code.

Переваги Visual Studio

- Підтримує безліч мов програмування.
- Ви можете змінити мову для вибраного файлу.
- Надає мовну документацію.
- Зручний інтерфейс
- Вбудована інтеграція Git.
- Палітра команд
- Широкі можливості кастомізації

Перше, що користувач бачить під час того як заходить на сайт це шапка сайту. HTML на прикладі шапки сайту.

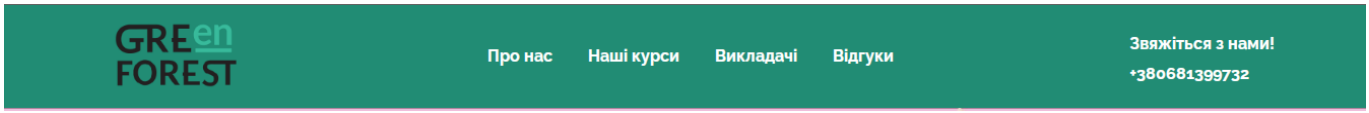

Рис. 3.4 Шапка сайту

Лістинг 1

```
<header class="header">
        <a class="logo">
            <img src="images/logo.png" alt="">
        </a>
        <nav class="links">
            <a href="#toAboutUs">Про нас</a>
            <a href="#toOurCourses">Наші курси</a>
            <a href="#toOurTeachers">Викладачі</a>
            <a href="#toOurReviews">Відгуки</a>
        </nav>
        <div class="contacts">
            <p>Звяжіться з нами!</p>
            <a href="tel:+380681399732">+380681399732</a>
        </div>
        <div class="nav-burger">
            <div></div>
            <div></div>
            <div></div>
        </div>
        <div class="side-menu">
            <a href="#toAboutUs">Про нас</a>
            <a href="#toOurCourses">Наші курси</a>
            <a href="#toOurTeachers">Викладачі</a>
            <a href="#toOurReviews">Відгуки</a>
            <br>
            <a href="tel:+380681399732">+380681399732</a>
            <img src="images/na_coli2.png" alt="">
        </div>
    </header>
```

CSS код розглянемо також на прикладі шапки сайту:

Лістинг 2

```
*{
    margin: 0px;
    padding: 0px;
    font-family: 'Raleway', sans-serif;
    box-sizing: border-box;
}
```

```
.header{
    height: 100px;
    width: 100%;
    background-color: #218c74;
    padding: 12px 96px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    z-index: 1000;
    position: relative;
}
.nav-burger{
    display: none;
    cursor: pointer;
}

.nav-burger div{
    width: 28px;
    height: 2px;
    background-color: #fbfbfb;
}

.nav-burger div:nth-of-type(2){
    margin: 8px 0;
}

.side-menu{
    display: none;
    height: 100vh;
    position: absolute;
    top: 100px;
    left: -100%;
    transition: all .65s;
    text-align: center;
    padding: 48px 0;
    background-color: #218c74;;
    width: 100%;
}

.side-menu a{
    display: block;
    font-size: 20px;
    text-decoration: none;
    line-height: 14px;
```

```
font-weight: 700;
text-transform: uppercase;
position: relative;
padding: 12px 0;
color: #fbfbfb;}
```

Також для сайту було розроблено адаптивну версію для всіх сучасних пристроїв (телефонів і планшетів). Адаптив виглядає наступним чином:

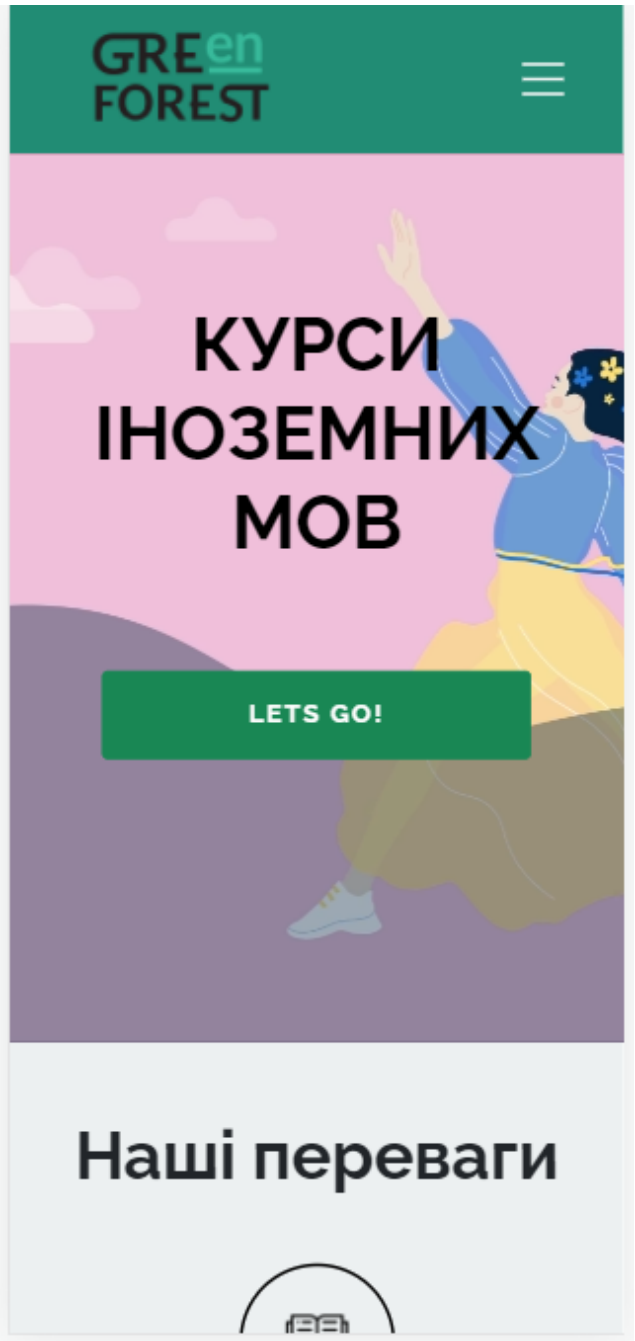

Рис. 3.5 Мобіль версія сайту при 414 x 896px

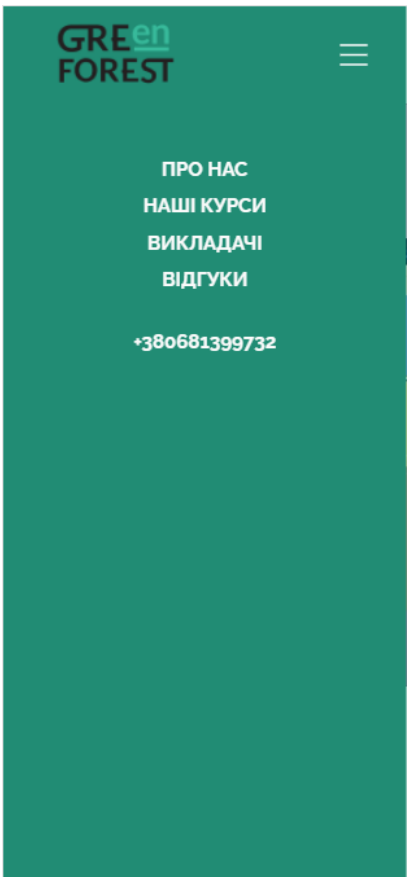

Рис. 3.6 Вигляд шапки сайту при адаптивній версії мобільних пристроїв 414 x 896px

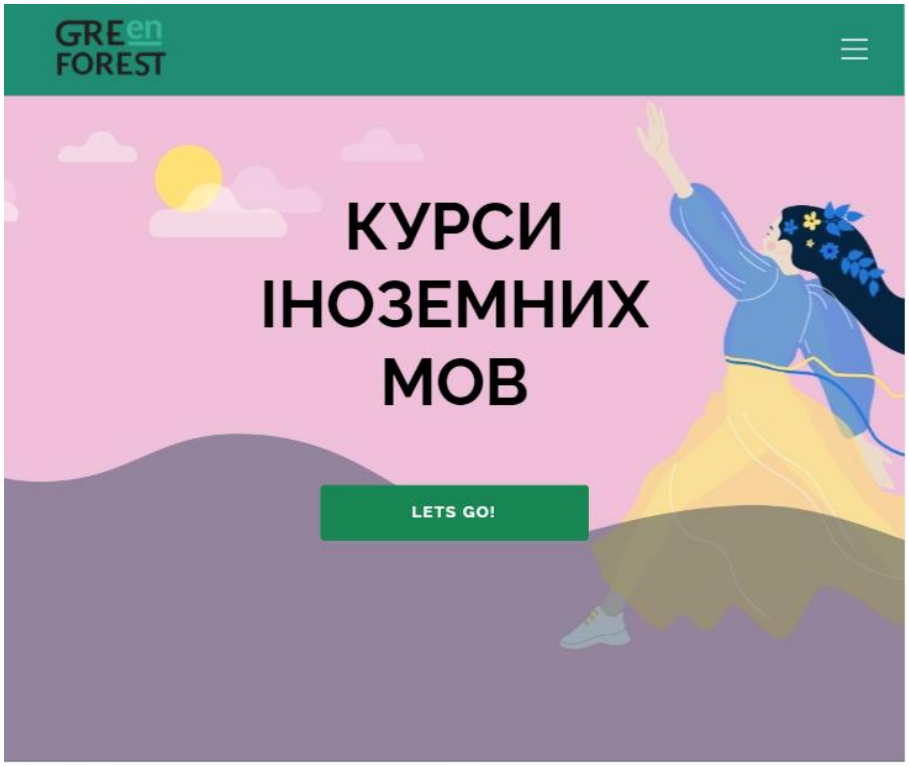

Рис. 3.7 Адаптивний вигляд сайту при 973x1189px

ВИСНОВКИ

В курсовій роботі було проаналізовано, як правильно використовувати MySql у роботі з даними, а також розглянуто особливості WEB розробки.

В процесі написання курсової роботи було виконано поставлені завдання, а саме:

- Розробка сайту з використанням сучасних технологій.
- Розробка бази даних і відповідних запитів які містять бізне-логіку
- Створення адаптивної версти та дизайну сайту.
- Розробка зручного та зрозумілого для користувача інтерфейсу.

Під час написання курсової використовувались наступні технології:

- HTML
- SQL
- BOOTSTRAP
- CSS

Було спроектовано та описано структуру БД, її таблиці та записи. Визначено типи даних та функціональні залежності. Розроблено запити та визначено атрибути полів. Для кращого розуміння структури бази даних побудовано ER діаграму. Вона показує взаємодію між відношеннями і дозволяє приступити до програмування бази даних.

Інформаційна система працює, адаптується, має зрозумілий для користувача інтерфейс. База даних є логічно пророблена, містить селекти які фільтрують і сортують інформацію в таблицях та показують результат користувачу. Дана інформаційна система має великий потенціал та може бути удосконалена в майбутньому.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Зображення в HTML [Електронний ресурс]. – Режим доступу: https://vertex-academy.com/tutorials/ru/izobrazheniya-v-html/
2. Бази даних та інформаційні системи — Вікі ЦДПУ [Електронний ресурс]. – Режим доступу: https://cutt.ly/CyTXnR1
3. КОРОТКЕ ОЗНАЙОМЛЕННЯ З ІНСТРУМЕНТОМ ДЛЯ БАЗ ДАНИХ – PHPMYADMIN | SEBWEO [Електронний ресурс]. – Режим доступу: 30 https://sebweo.com/korotke-oznajomlennya-z-instrumentom-dlya-baz-danihphpmyadmin/
4. Вікіпедія. [Електронний ресурс]. - Режим доступу: https://uk.wikipedia.org
5. Cpp-reference. [Електронний ресурс]. – Режим доступу : http://cppreference.ru/patterns/behavioral-patterns/command/
6. Bootstrap [Електронний ресурс]. – Режим доступу: https://timeweb.com/ru/community/articles/plyusy-i-minusy-bootstrap-1.
7. ОСНОВИ БАЗ ДАНИХ [Електронний ресурс]. – Режим доступу: http://www.kievoit.ippo.kubg.edu.ua/kievoit/2013/118/118.html
8. Бази даних та інформаційні системи — Вікі ЦДПУ [Електронний ресурс]. – Режим доступу: https://cutt.ly/CyTXnR1
9. БАЗЫ ДАННЫХ - УРОК 3. РЕЛЯЦИОННЫЕ БАЗЫ ДАННЫХ [Електронний ресурс]. – Режим доступу: https://site-do.ru/db/db3.php

ДОДАТКИ

Додаток А

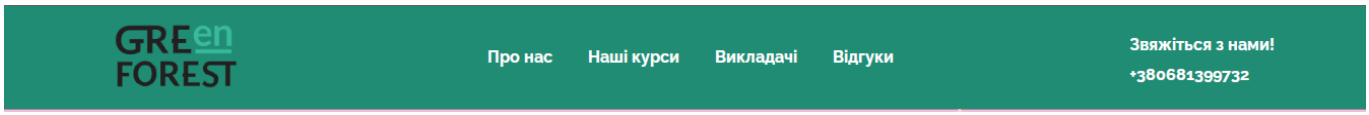

Рис. 3.1 Хедер сайта

Додаток Б

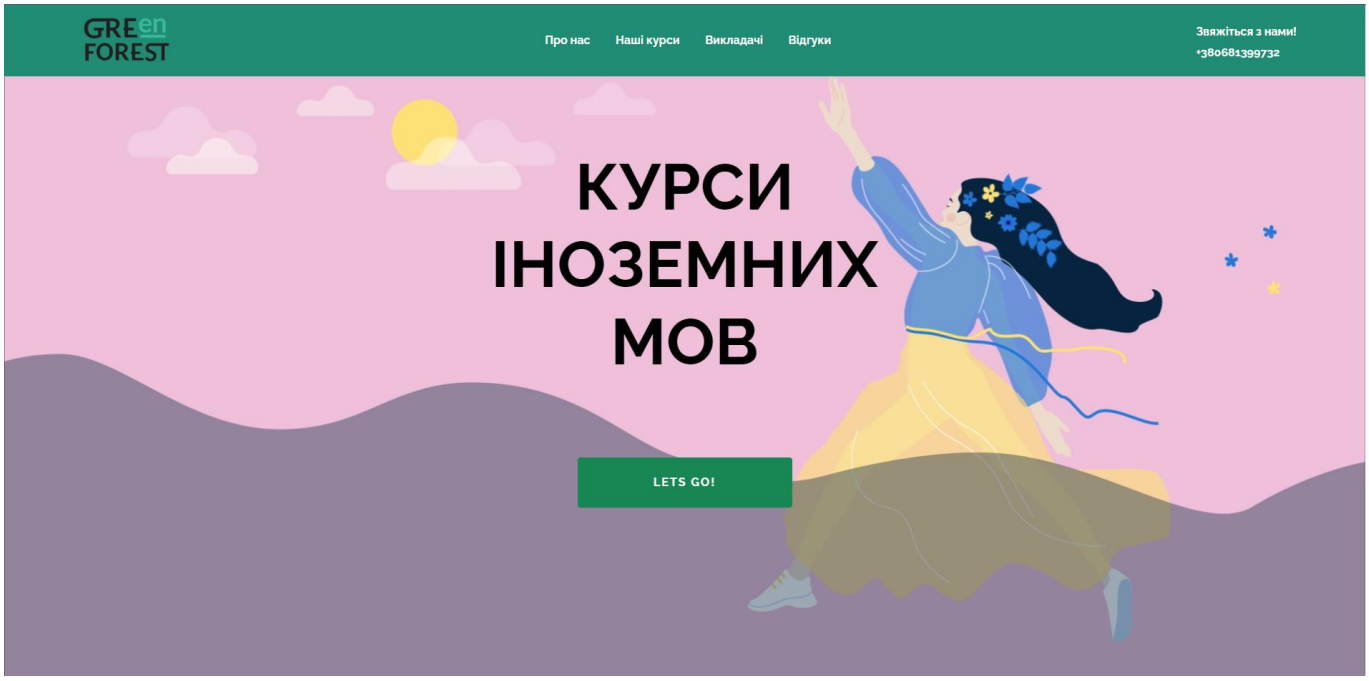

Рис. 3.2 Фронтальна секція

Додаток В

Рис. 3.3

Додаток Г

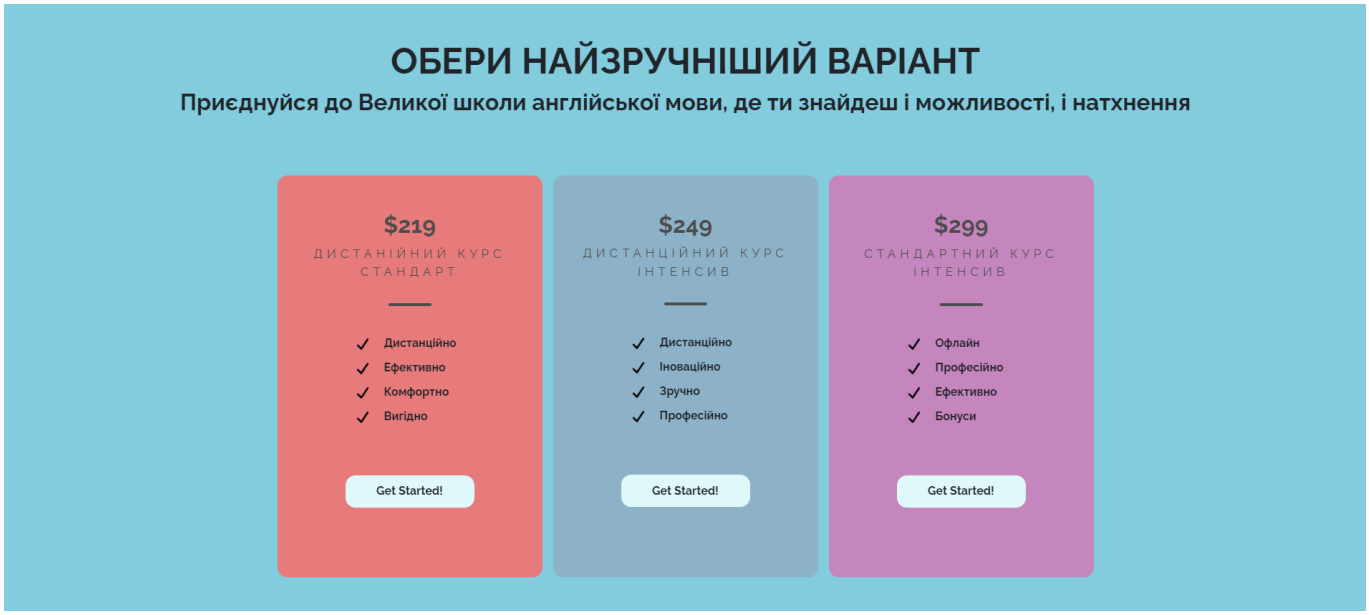

Рис. 3.4 Перелік курсів

Додаток Ґ

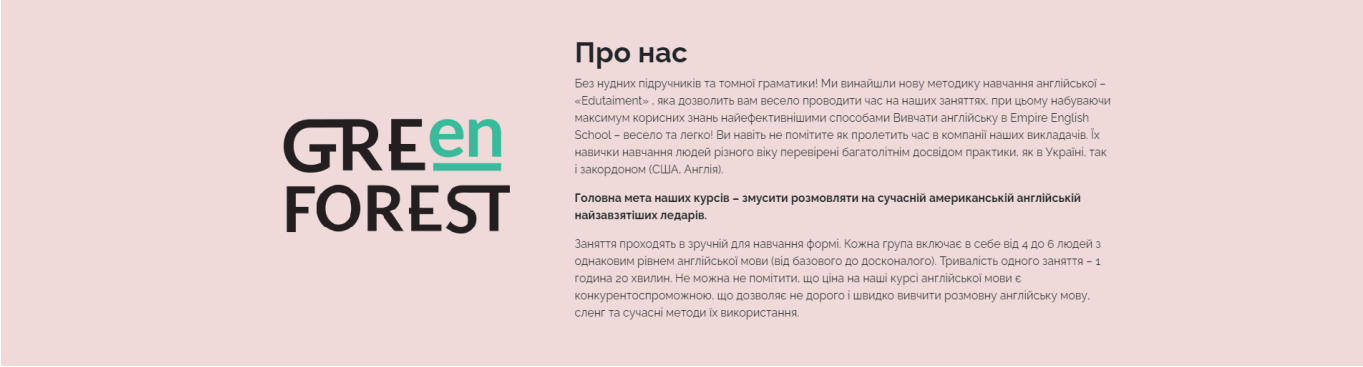

Рис. 3.5 Секція «Про нас»

Додаток Д

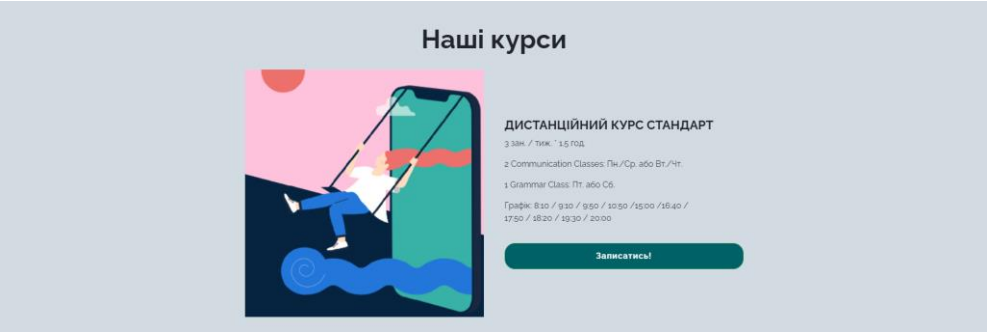

Рис. 3.6 Детальна інформація про кожен курс

Додаток Е

Рис. 3.7 Секція «Викадачі»

Додаток Є

Рис. 3.8 Секція «Відгуки»

Додаток Ж

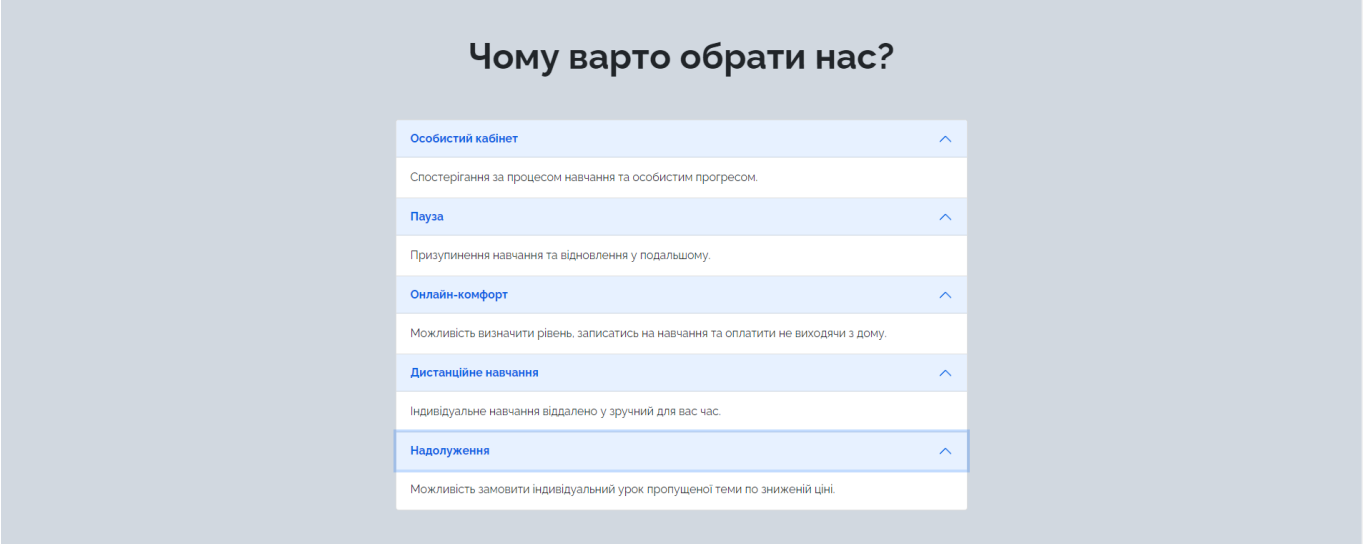

Рис. 3.9 Секція «Чому варто обрати нас»

Додаток З

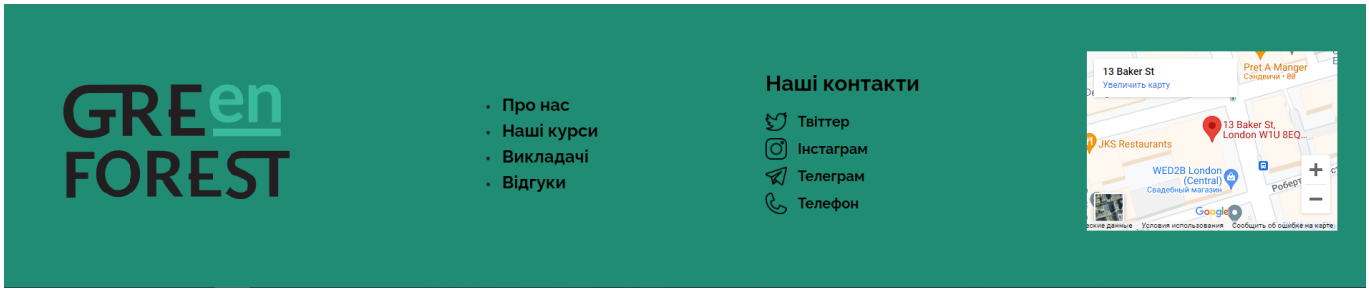

Рис. 3.10 Футер

Додаток И Лістинг Бази Даних

Створення бази даних і таблиць

```
CREATE DATABASE COURSE;


USE COURSE;
```

CREATE TABLE LESSONS (ID INT NOT NULL, CLASSROMMID INT NOT NULL, COURSEID INT NOT NULL, TEACHERID INT NOT NULL, chas VARCHAR(30), WEEKDAY VARCHAR(50));

CREATE TABLE COURSES (ID INT NOT NULL, COURSENAME VARCHAR(30), DESCRIPTION_COURSE VARCHAR(100), CATEGORY VARCHAR(30), PRICE INT);

CREATE TABLE STUDENT_GROUP (ID INT NOT NULL, CLASSID INT NOT NULL, STUDENTID INT NOT NULL);

CREATE TABLE STUDENT (ID INT NOT NULL, FIRSTNAME VARCHAR(30), LASTTNAME VARCHAR(30), DATE_OF_BIRTH DATE, YEAR_GROUP varchar (50), GENDER VARCHAR(30), EMAIL VARCHAR(50), PHONE varchar(30));

CREATE TABLE TEACHER (ID INT NOT NULL, FIRSTNAME VARCHAR(30), LASTTNAME VARCHAR(30), ranks VARCHAR(50), languag VARCHAR(50), EMAIL VARCHAR(50), PHONE varchar(30), SLARY int not null, DATE_EMPLOYMENT date) ;

CREATE TABLE CLASS_ROOM (ID INT NOT NULL, ROOMTYPE VARCHAR(30), FLOOR INT);

CREATE TABLE PAYMENT (ID INT NOT NULL, PAYMENT_DATE date, PAYMENT_METHOD_ID INT, STAT VARCHAR(50), STUDENT_ID INT NOT NULL, COURSEID INT NOT NULL);

CREATE TABLE PAYMENT_METHOD (ID INT NOT NULL, NAME VARCHAR(50));

Створення зв'язків між таблицями

ALTER TABLE COURSES ADD CONSTRAINT COURSEID PRIMARY KEY (ID);

ALTER TABLE LESSONS ADD FOREIGN KEY (COURSEID) REFERENCES COURSES(ID);

ALTER TABLE CLASS_ROOM ADD CONSTRAINT CLASSROMMID PRIMARY KEY (ID);

ALTER TABLE LESSONS ADD FOREIGN KEY (CLASSROMMID) REFERENCES CLASS_ROOM(ID);

ALTER TABLE TEACHER ADD CONSTRAINT TEACHERID PRIMARY KEY (ID);

ALTER TABLE LESSONS ADD FOREIGN KEY (TEACHERID) REFERENCES TEACHER(ID);

ALTER TABLE STUDENT ADD CONSTRAINT STUDENTID PRIMARY KEY (ID);

ALTER TABLE STUDENT_GROUP ADD FOREIGN KEY (STUDENTID) REFERENCES STUDENT(ID);

ALTER TABLE LESSONS ADD CONSTRAINT CLASSID PRIMARY KEY (ID);

ALTER TABLE STUDENT_GROUP ADD FOREIGN KEY (CLASSID) REFERENCES LESSONS(ID);

ALTER TABLE PAYMENT ADD FOREIGN KEY (STUDENT_ID) REFERENCES STUDENT(ID);

ALTER TABLE PAYMENT ADD FOREIGN KEY (COURSEID) REFERENCES COURSES(ID);

ALTER TABLE PAYMENT_METHOD ADD CONSTRAINT PAYMENT_METHOD_ID PRIMARY KEY (ID);

ALTER TABLE PAYMENT ADD FOREIGN KEY (PAYMENT_METHOD_ID) REFERENCES PAYMENT_METHOD(ID);

Наповнення таблиць даними

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (1, 'Oleh', 'Semkiv', '1992-09-13', '20-30', 'male', 'olegsemkiv@gmail.com', 380685399731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (2, 'Romanna', 'Ryvko', '2003-11-22', '16-20', 'famale', 'romannaryvko@gmail.com', 380681356731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (3, 'Bohdan', 'Felypets', '1995-01-12', '20-30', 'male', 'bohdanfelypets@gmail.com', 380661399731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (4, 'Yaroslav', 'Yarynych', '2002-04-15', '16-20', 'male', 'yaroslavyarynych@gmail.com', 380681366731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (5, 'Sergiy', 'Tekh', '1999-12-17', '16-20', 'male', 'sergiytekh@gmail.com', 380684449731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (6, 'Maria', 'Lenko', '2002-12-19', '16-20', 'famale', 'marialenko@gmail.com', 380685399731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (7, 'Roman', 'Kuzma', '2001-07-20', '20-30', 'male', 'romankuzma@gmail.com', 380681356731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (8, 'Julia', 'Bil', '1996-04-24', '20-30', 'famale', 'juliabill@gmail.com', 380661399731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (9, 'Yaroslav', 'Mudryi', '2004-03-08', '16-20', 'male', 'yaroslavmudruy@gmail.com', 380231366731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (10, 'Oleg', 'Barylko', '2002-03-08', '16-20', 'male', 'olegbarylko@gmail.com', 380244366731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (11, 'Marta', 'Lushch', '2001-05-04', '16-20', 'famale', 'martalushch@gmail.com', 380684099731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (12, 'Alina', 'Babii', '1998-05-05', '20-30', 'famale', 'alinababii@gmail.com', 380984449731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (13, 'Anhelina', 'Vasechko', '2005-02-15', '16-20', 'famale', 'anhelinavasechko@gmail.com', 38674449731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (14, 'Solomiya', 'Shevchuk', '2006-03-17', '14-16', 'famale', 'solomiyashevchuk@gmail.com', 380564449731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (15, 'Oksana', 'Gula', '1990-04-18', '20-30', 'famale', 'oksanagula@gmail.com', 380684239731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (16, 'Stepan', 'Struk', '2005-11-18', '16-20', 'male', 'stepanstruk@gmail.com', 380684467731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (17, 'Roman', 'Romanov', '1990-06-05', '20-30', 'male', 'romanromanov@gmail.com', 380655449731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (18, 'Roman', 'Semkiv', '2006-05-12', '14-16', 'male', 'romansemkiv@gmail.com', 380684449731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (19, 'Yevgen', 'Osoba', '1994-04-14', '20-30', 'male', 'yevgenosoba@gmail.com', 380224449731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (20, 'Nazar', 'Didus', '2007-09-03', '14-16', 'male', 'nazardidus@gmail.com', 380684779731);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (21, 'Taras', 'Shevchenko', '2003-08-05', '20-30', 'male', 'tarasshevchenko@gmail.com', 380975534223);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (22, 'Ihor', 'Lys', '1999-09-03', '14-16', 'male', 'ihorlys@gmail.com', 380654356778);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (23, 'Dana', 'Zayushkina', '1990-12-12', '16-20', 'famale', 'zayushkina@gmail.com', 380986566887);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (24, 'Marta', 'Kalnyk', '2000-01-03', '20-30', 'male', 'martakalnyk@gmail.com', 380973212445);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (25, 'Oleksa', 'Kravets', '2006-10-10', '14-16', 'male', 'oleksa@gmail.com', 380675578954);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (26, 'Olha', 'Koval', '2000-11-03', '20-30', 'famale', 'olha@gmail.com', 380973322555);

INSERT INTO student (ID , FIRSTNAME, LASTTNAME, DATE_OF_BIRTH , YEAR_GROUP , GENDER , EMAIL , PHONE) valueS (27, 'Roman', 'Petrakov', '1998-01-15', '14-16', 'male', 'petrakov@gmail.com', 380976654223);

SELECT * FROM student;

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE, slary, DATE_EMPLOYMENT) VALUES (1,'Denys', 'Malushko', 'A2', 'ENGLISH', 'denysmalushko@gmail.com', '380681399445', 15000, '2018-09-03');

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE, slary , DATE_EMPLOYMENT) VALUES (2,'Olena', 'Meshenina', 'B2', 'GERMANY', 'olenameshenina@gmail.com', '380681399321', 16000, '2019-09-03');

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE, slary , DATE_EMPLOYMENT) VALUES (3,'Oleg', 'Maletskuy', 'C1', 'ENGLISH', 'olegmaletskuy@gmail.com', '380681399112', 15000, '2015-09-03');

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE , slary, DATE_EMPLOYMENT) VALUES (4,'Julia', 'Teslyuk', 'C1', 'SPANISH', 'juliateslyuk@gmail.com', '380681399731', 18000, '2016-09-03');

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE, slary , DATE_EMPLOYMENT) VALUES (5,'Nazar', 'Teslyuk', 'B2', 'JAPAN', 'nazarteslyuk@gmail.com', '380681399554', 20000, '2016-09-03');

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE , slary, DATE_EMPLOYMENT) VALUES (6,'Iruna', 'Begun', 'C2', 'ENGLISH', 'irunabegun@gmail.com', '380681399777', 15000, '2020-09-03');

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE , slary, DATE_EMPLOYMENT) VALUES (7,'Lesya', 'Grycevych', 'B2', 'JAPAN', 'lesyagrycevych@gmail.com', '380681399889', 20000, '2019-09-03');

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE , slary, DATE_EMPLOYMENT) VALUES (8,'Iryna', 'Topolska', 'C1', 'JAPAN', 'irynatopolska@gmail.com', '380681399009', 20000, '2019-09-03');

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE , slary, DATE_EMPLOYMENT) VALUES (9,'Oksana', 'Seminska', 'C2', 'UKRAINE', 'oksanaseminska@gmail.com', '380681399667', 17000, '2021-09-03');

INSERT INTO TEACHER (ID , FIRSTNAME , LASTTNAME , ranks , languag, EMAIL , PHONE , slary, DATE_EMPLOYMENT) VALUES (10,'Ivan', 'Kalnyk', 'C2', 'UKRAINE', 'ivankalnyk@gmail.com', '380681399889', 19000, '2020-09-03');

SELECT * FROM teacher;

INSERT INTO CLASS_ROOM (ID, ROOMTYPE, FLOOR) VALUES (1, "conference", 1);

INSERT INTO CLASS_ROOM (ID, ROOMTYPE, FLOOR) VALUES (2, "lesson_room", 2);

INSERT INTO CLASS_ROOM (ID, ROOMTYPE, FLOOR) VALUES (3, "conference", 3);

INSERT INTO CLASS_ROOM (ID, ROOMTYPE, FLOOR) VALUES (4, "lesson_room", 4);

```
INSERT INTO CLASS_ROOM (ID, ROOMTYPE, FLOOR) VALUES (5, "conference", 5);

INSERT INTO CLASS_ROOM (ID, ROOMTYPE, FLOOR) VALUES (6, "lesson_room", 6);

INSERT INTO CLASS_ROOM (ID, ROOMTYPE, FLOOR) VALUES (7, "online", 0);


SELECT * FROM CLASS_ROOM;


INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE ) VALUES (1, 'Курс Англійської ОНЛАЙН', 'ENGLISH_course_description', 'ONLINE', 5000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE ) VALUES (2, 'Курс Німецької ОНЛАЙН', 'GERMANY_course_description', 'ONLINE', 4500);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE ) VALUES (3, 'Курс Японської ОНЛАЙН', 'JAPAN_course_description', 'ONLINE', 6000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE ) VALUES (4, 'Курс Іспанської ОНЛАЙН', 'SPANISH_course_description', 'ONLINE', 7800);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE ) VALUES (5, 'Курс Української ОНЛАЙН', 'UKRAINE_course_description', 'ONLINE', 4750);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE ) VALUES (6, 'Курс Англійської', 'ENGLISH_course_description', 'OFFLINE', 10000);
```

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (7, 'Курс Німецької', 'germany_course_description', 'OFFLINE', 9000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (8, 'Курс Японської', 'JAPAN_course_description', 'OFFLINE', 12000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (9, 'Курс Іспанської', 'SPANISH_course_description', 'OFFLINE', 15000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (10, 'Курс Української', 'UKRAINE_course_description', 'OFFLINE', 9500);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (11, 'Курс Англійської Розмовний', 'ENGLISH_course_description', 'ONLINE', 4500);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (12, 'Курс Німецької Розмовний', 'GERMANY_course_description', 'ONLINE', 5000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (13, 'Курс Японської Розмовний', 'JAPAN_course_description', 'ONLINE', 5000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (14, 'Курс Іспанської Розмовний', 'SPANISH_course_description', 'ONLINE', 4500);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (15, 'Курс Української Розмовний', 'UKRAINE_course_description', 'ONLINE', 4000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (16, 'Курс Англійської Граматика', 'ENGLISH_course_description', 'OFFLINE', 6000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (17, 'Курс німецької Граматика', 'germany_course_description', 'OFFLINE', 5000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (18, 'Курс Японської Граматика', 'JAPAN_course_description', 'OFFLINE', 6000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (19, 'Курс Іспанської Граматика', 'SPANISH_course_description', 'OFFLINE', 10000);

INSERT INTO COURSES (ID, COURSENAME, DESCRIPTION_COURSE, CATEGORY, PRICE) VALUES (20, 'Курс Української Граматика', 'UKRAINE_course_description', 'OFFLINE', 5000);

SELECT * FROM COURSES;

INSERT INTO LESSONS (ID, CLASSROMMID, COURSEID, TEACHERID, chas, WEEKDAY) VALUES (1,7,1,1,'12-00', 'Понеділок');

INSERT INTO LESSONS (ID, CLASSROMMID, COURSEID, TEACHERID, chas, WEEKDAY) VALUES (2,7,2,2,'11-00', 'Вівторок');

INSERT INTO LESSONS (ID, CLASSROMMID, COURSEID, TEACHERID, chas, WEEKDAY) VALUES (3,7,3,3,'15-00', 'Середа');

INSERT INTO LESSONS (ID, CLASSROMMID, COURSEID, TEACHERID, chas, WEEKDAY) VALUES (4,7,4,4,'13-00', 'Четвер');

INSERT INTO LESSONS (ID, CLASSROMMID, COURSEID, TEACHERID, chas, WEEKDAY) VALUES (5,7,5,5,'18-00', 'Пятниця');

INSERT INTO LESSONS (ID, CLASSROMMID, COURSEID, TEACHERID, chas, WEEKDAY) VALUES (6,1,6,1,'14-00', 'Понеділок');

INSERT INTO LESSONS (ID, CLASSROMMID, COURSEID, TEACHERID, chas, WEEKDAY) VALUES (7,2,7,2,'13-00', 'Вівторок');

INSERT INTO LESSONS (ID, CLASSROMMID, COURSEID, TEACHERID, chas, WEEKDAY) VALUES (8,3,8,3,'17-00', 'Середа');

INSERT INTO LESSONS (ID, CLASSROMMID, COURSEID, TEACHERID, chas, WEEKDAY) VALUES (9,5,9,4,'16-00', 'Четвер');

SELECT * FROM LESSONS;

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID) VALUES (1,1,1);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID) VALUES (2,1,2);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID) VALUES (3,1,3);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID) VALUES (4,1,4);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID) VALUES (5,2,5);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID) VALUES (6,2,6);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID) VALUES (7,2,7);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID) VALUES (8,2,8);

```
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (9,3,9);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (10,3,10);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (11,3,11);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (12,3,12);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (13,4,13);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (14,4,14);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (15,4,15);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (16,4,16);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (17,5,17);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (18,5,18);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (19,5,19);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (20,5,20);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (21,5,17);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (22,5,18);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (23,5,19);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (24,5,20);

INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (25,5,17);
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (26,5,18);
```

```
INSERT INTO STUDENT_GROUP(ID, CLASSID, STUDENTID)  VALUES (27,5,19);



SELECT * FROM STUDENT_GROUP;


INSERT INTO PAYMENT_METHOD(ID, NAME) VALUES (1, 'CASH');

INSERT INTO PAYMENT_METHOD(ID, NAME) VALUES (2, 'CARD');


SELECT * FROM PAYMENT_METHOD;


INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT,
STUDENT_ID, COURSEID) VALUES (1,'2022-01-11',  1, 'COMING',1,1);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT,
STUDENT_ID, COURSEID) VALUES (2,'2022-01-11',  1, 'COMING',1,2);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT,
STUDENT_ID, COURSEID) VALUES (3,'2022-01-11',  1, 'COMING',1,3);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT,
STUDENT_ID, COURSEID) VALUES (4,'2022-02-15',  1, 'COMPLETE',1,2);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT,
STUDENT_ID, COURSEID) VALUES (5,'2022-03-17',  1, 'COMPLETE',3,3);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT,
STUDENT_ID, COURSEID) VALUES (6,'2022-03-17',  1, 'COMPLETE',3,4);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT,
STUDENT_ID, COURSEID) VALUES (7,'2022-04-05',  1, 'COMING',4,5);
```

```
INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (8,'2022-04-05',  1, 'COMING',4,6);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (9,'2022-04-05',  1, 'COMING',4,1);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (10,'2022-04-05',  1, 'COMING',4,2);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (11,'2022-05-01',  1, 'COMING',5,7);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (12,'2022-07-19',  1, 'COMPLETE',6,8);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (13,'2022-11-27',  1, 'COMPLETE',7,9);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (14,'2022-12-18',  1, 'COMING',8,10);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (15,'2022-12-16',  1, 'COMING',9,10);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (16,'2022-10-11',  1, 'COMING',10,2);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (17,'2022-09-11',  2, 'COMPLETE',11,3);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (18,'2022-01-15',  2, 'COMING',12,4);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (19,'2022-05-03',  2, 'COMPLETE',13,4);

INSERT INTO PAYMENT(ID, PAYMENT_DATE,  PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (20,'2022-11-06',  2, 'COMING',14,5);
```

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (21,'2022-05-12', 2, 'COMPLETE',15,7);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (22,'2022-12-13', 2, 'COMING',16,8);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (23,'2022-12-19', 2, 'COMPLETE',17,4);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (24,'2022-05-20', 2, 'COMING',18,5);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (25,'2022-10-21', 2, 'COMPLETE',19,8);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (26,'2022-11-07', 2, 'COMPLETE',21,5);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (27,'2022-11-08', 2, 'COMPLETE',22,6);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (28,'2022-11-09', 2, 'COMPLETE',23,6);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (29,'2022-11-10', 2, 'COMING',24,6);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (30,'2022-11-11', 2, 'COMPLETE',25,10);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (31,'2022-11-12', 2, 'COMPLETE',26,5);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (32,'2022-11-13', 2, 'COMPLETE',26,9);

INSERT INTO PAYMENT(ID, PAYMENT_DATE, PAYMENT_METHOD_ID, STAT, STUDENT_ID, COURSEID) VALUES (33,'2022-11-15', 2, 'COMPLETE',27,9);

Створення запитів

/*сортування за віковою категорією */

SELECT * FROM STUDENT WHERE YEAR_GROUP = '16-20';

SELECT * FROM STUDENT WHERE YEAR_GROUP = '20-30';

SELECT * FROM STUDENT WHERE YEAR_GROUP = '14-16';

/*сортування за гендером */

SELECT * FROM STUDENT WHERE GENDER = 'MALE';

SELECT * FROM STUDENT WHERE GENDER = 'FAMALE';

/*сортування викладачів по мові */

SELECT FIRSTNAME ,LASTTNAME, RANKS FROM TEACHER WHERE LANGUAG = 'UKRAINE';

SELECT FIRSTNAME ,LASTTNAME, RANKS FROM TEACHER WHERE LANGUAG = 'ENGLISH';

SELECT FIRSTNAME ,LASTTNAME, RANKS FROM TEACHER WHERE LANGUAG = 'JAPAN';

SELECT FIRSTNAME ,LASTTNAME, RANKS FROM TEACHER WHERE LANGUAG = 'SPANISH';

SELECT FIRSTNAME ,LASTTNAME, RANKS FROM TEACHER WHERE LANGUAG = 'GERMANY';

/*розклад занять */

SELECT LESSONS.ID, teacher.firstname, teacher.lasttname, COURSENAME, LESSONS.weekday, lessons.chas, CLASS_ROOM.ROOMTYPE, CLASS_ROOM.FLOOR FROM LESSONS LEFT JOIN teacher on lessons.teacherid = teacher.id

LEFT JOIN COURSES ON LESSONS.COURSEID = COURSES.ID

LEFT JOIN CLASS_ROOM ON LESSONS.CLASSROMMID = CLASS_ROOM.ID ;

/*риозклад по днях */

SELECT LESSONS.ID, teacher.firstname, teacher.lasttname, COURSENAME, LESSONS.weekday, lessons.chas, CLASS_ROOM.ROOMTYPE, CLASS_ROOM.FLOOR FROM LESSONS LEFT JOIN teacher on lessons.teacherid = teacher.id

LEFT JOIN COURSES ON LESSONS.COURSEID = COURSES.ID

LEFT JOIN CLASS_ROOM ON LESSONS.CLASSROMMID = CLASS_ROOM.ID WHERE weekday = 'Понеділок';

SELECT LESSONS.ID, teacher.firstname, teacher.lasttname, COURSENAME, LESSONS.weekday, lessons.chas, CLASS_ROOM.ROOMTYPE, CLASS_ROOM.FLOOR FROM LESSONS LEFT JOIN teacher on lessons.teacherid = teacher.id

LEFT JOIN COURSES ON LESSONS.COURSEID = COURSES.ID

LEFT JOIN CLASS_ROOM ON LESSONS.CLASSROMMID = CLASS_ROOM.ID WHERE weekday = 'Вівторок';

SELECT LESSONS.ID, teacher.firstname, teacher.lasttname, COURSENAME, LESSONS.weekday, lessons.chas, CLASS_ROOM.ROOMTYPE, CLASS_ROOM.FLOOR FROM LESSONS LEFT JOIN teacher on lessons.teacherid = teacher.id

LEFT JOIN COURSES ON LESSONS.COURSEID = COURSES.ID

LEFT JOIN CLASS_ROOM ON LESSONS.CLASSROMMID = CLASS_ROOM.ID WHERE weekday = 'Середа';

SELECT LESSONS.ID, teacher.firstname, teacher.lasttname, COURSENAME, LESSONS.weekday, lessons.chas, CLASS_ROOM.ROOMTYPE, CLASS_ROOM.FLOOR FROM LESSONS LEFT JOIN teacher on lessons.teacherid = teacher.id

LEFT JOIN COURSES ON LESSONS.COURSEID = COURSES.ID

LEFT JOIN CLASS_ROOM ON LESSONS.CLASSROMMID = CLASS_ROOM.ID WHERE weekday = 'Четвер';

SELECT LESSONS.ID, teacher.firstname, teacher.lasttname, COURSENAME, LESSONS.weekday, lessons.chas, CLASS_ROOM.ROOMTYPE, CLASS_ROOM.FLOOR FROM LESSONS LEFT JOIN teacher on lessons.teacherid = teacher.id

LEFT JOIN COURSES ON LESSONS.COURSEID = COURSES.ID

LEFT JOIN CLASS_ROOM ON LESSONS.CLASSROMMID = CLASS_ROOM.ID WHERE weekday = 'Пятниця';

SELECT LESSONS.ID, teacher.firstname, teacher.lasttname, COURSENAME, LESSONS.weekday, lessons.chas, CLASS_ROOM.ROOMTYPE,

```
CLASS_ROOM.FLOOR FROM LESSONS LEFT JOIN teacher  on  lessons.teacherid =
teacher.id

LEFT JOIN COURSES ON LESSONS.COURSEID = COURSES.ID

LEFT JOIN CLASS_ROOM ON LESSONS.CLASSROMMID = CLASS_ROOM.ID
WHERE weekday = 'Субота';


/*Студенти які купили більше 1 курсу */

SELECT STUDENT.ID, STUDENT.FIRSTNAME, STUDENT.LASTTNAME

 FROM STUDENT left join PAYMENT on PAYMENT.STUDENT_ID = STUDENT.ID

 group by STUDENT.id having count(COURSEID) > 1;


/*Викладачі які викладають більше 1 курсу */

SELECT TEACHER.ID, TEACHER.FIRSTNAME, TEACHER.LASTTNAME

 FROM TEACHER left join LESSONS on LESSONS.TEACHERID = TEACHER.ID

 group by TEACHER.id having count(COURSEID) > 1;


 /*список студентів які оплатили готівкою */

SELECT PAYMENT.ID,  STUDENT.firstname, STUDENT.lasttname,
PAYMENT_METHOD.NAME FROM PAYMENT LEFT JOIN STUDENT  on
PAYMENT.STUDENT_ID = STUDENT.id

LEFT JOIN PAYMENT_METHOD ON PAYMENT.PAYMENT_METHOD_ID =
PAYMENT_METHOD.ID WHERE PAYMENT_METHOD_ID = 1;


 /*список студентів які оплатили картою */
```

```
SELECT PAYMENT.ID,  STUDENT.firstname, STUDENT.lasttname,
PAYMENT_METHOD.NAME FROM PAYMENT LEFT JOIN STUDENT  on
PAYMENT.STUDENT_ID = STUDENT.id

LEFT JOIN PAYMENT_METHOD ON PAYMENT.PAYMENT_METHOD_ID =
PAYMENT_METHOD.ID WHERE PAYMENT_METHOD_ID = 2;



 /*список студентів які успішно оплатили курс*/

 SELECT PAYMENT.ID,  STUDENT.firstname, STUDENT.lasttname,
PAYMENT.STAT FROM PAYMENT LEFT JOIN STUDENT  on
PAYMENT.STUDENT_ID = STUDENT.id

 WHERE STAT = 'COMPLETE';


 /*найдорожчий і найдешевший курс */

SELECT max(price) from courses;

SELECT min(price) from courses;


 /*сортування від найдорожчого / найдешевшого курсу */

 SELECT *FROM courses ORDER BY price DESC;

 SELECT *FROM courses ORDER BY price asc;


 /*список викладачів які працюють від 2017 і від 2019 */

SELECT id, firstname, lasttname, slary FROM TEACHER WHERE
YEAR(DATE_EMPLOYMENT) < 2017;
```

```
SELECT id, firstname, lasttname, slary FROM TEACHER WHERE
YEAR(DATE_EMPLOYMENT) > 2019;


 /*список студентів для нарахування знижки */

SELECT STUDENT.ID, STUDENT.FIRSTNAME, STUDENT.LASTTNAME,
COURSES.price as sum

FROM STUDENT left join PAYMENT on PAYMENT.STUDENT_ID = STUDENT.ID

left JOIN COURSES ON PAYMENT.COURSEID = courses.id

group by STUDENT.id having sum(price) > 10000;


 /*список викладачів для нарахування премії */

SELECT TEACHER.ID, TEACHER.FIRSTNAME, TEACHER.LASTTNAME,
COURSES.price as sum

 FROM TEACHER left join LESSONS on LESSONS.TEACHERID = TEACHER.ID

 left JOIN COURSES ON LESSONS.COURSEID = courses.id

 group by TEACHER.id having sum(price) > 7000;


 /*загальна статистика продажу курсів */

select courses.id as 'АЙДІ' , courses.coursename as КУРС, count(payment.id) as
КІЛЬКІСТЬ, SUM(COURSES.PRICE) as СУМА

from courses join payment on courses.id = payment.courseid

where payment.stat like 'COMPLETE'

GROUP BY COURSES.ID order by СУМА;
```

```
 /*загальний прибуток */
select  count(payment.id) as КІЛЬКІСТЬ, SUM(COURSES.PRICE) as СУМА
from courses join payment on courses.id = payment.courseid
where payment.stat like 'COMPLETE';


 /*прибуток який очікується */
select  count(payment.id) as КІЛЬКІСТЬ, SUM(COURSES.PRICE) as СУМА
from courses join payment on courses.id = payment.courseid
where payment.stat like 'coming';
```